



UNIVERSITÉ DE VERSAILLES SAINT-QUENTIN-EN-YVELINES UNIVERSITÉ PARIS-SACLAY

UFR des Sciences – Master 1 E3A

RAPPORT DE PROJET DE COLOCATION

Apprentissage de séries temporelles à l'aide de réseaux de neurones

Réalisé par :

- M. Mohamed OTMANI
- M. Mohammed BOUKACHABIA

Encadrant de projet : M. Aymeric CHAZOTTES

Jury : L. Barthes, A. Chazottes, N. Oudart

Organisme d'accueil : Laboratoire Atmosphères, Observations Spatiales (LATMOS)

Année universitaire : 2025 – 2026

Chapitre 1 : Introduction.....	2
1.1 Contexte et objectifs.....	2
1.2 Présentation du laboratoire LATMOS.....	3
Chapitre 2 : Données et ensembles.....	4
2.1 Nature et structure des données instrumentales.....	4
2.2 Stratégie de découpage temporel (fenêtrage).....	6
2.3 Partition des ensembles d'apprentissage, validation et test.....	7
2.4 Pipeline d'ingestion logicielle (Dataset et DataLoader PyTorch).....	7
Chapitre 3 : Auto-encodeur convolutif 1D (CAE).....	8
3.1 Principe des auto-encodeurs.....	8
3.4 Implémentation du CAE 1D.....	9
Chapitre 4 : Analyse des résultats.....	12
4.1 Démarche expérimentale.....	12
4.2 Configurations.....	12
4.3 Extension à deux voies.....	15
4.4 Analyse de l'indépendance des variables latentes (modèle deux voies).....	16
4.5 Bilan des résultats.....	17
Chapitre 5 : Conclusion et perspectives.....	18
5.1 Bilan du projet.....	18
5.2 Limites et difficultés rencontrées.....	18
5.3 Perspectives.....	18
Bibliographie.....	20
Annexe A : Rappels sur l'apprentissage profond.....	21
A.1 Architecture d'un réseau de neurones.....	21
A.2 Fonction de perte et optimisation.....	22
A.3 Rétropropagation du gradient.....	23
Annexe B : Réseaux de neurones convolutifs (CNN).....	25
B.1 Principe général.....	25
B.2 Convolution 2D (images) et 1D (séries temporelles).....	25
B.3 Architecture classique d'un CNN.....	27
B.4 Pourquoi les CNN 1D sont adaptés aux séries temporelles.....	28
B.5 Apprentissage des filtres.....	29
B.6 Exemple d'architecture CNN 1D simple.....	29
Annexe C : Réseaux de neurones récurrents (RNN) pour la dynamique temporelle.....	30
C.1 Limites des CNN pour les séquences longues.....	30
C.2 Principe du RNN et mémoire temporelle.....	30
C.3 Problèmes de gradient (vanishing / exploding).....	31
C.4 Architectures avancées : LSTM et GRU.....	31
C.5 Application envisagée dans le projet.....	32
C.6 Contraintes et perspectives.....	32
Annexe D : Codes sources.....	34

Chapitre 1 : Introduction

1.1 Contexte et objectifs

La caractérisation microphysique des précipitations constitue un enjeu fondamental en météorologie. Pour y répondre, le Laboratoire LATMOS utilise un disdromètre optique bifaisceau (DBS pour l'anglais Dual Beam Spectropluviomètre) capable de capturer à très haute fréquence les signaux générés par le passage des gouttes de pluie à travers deux faisceaux laser parallèles. Chaque événement pluvieux produit des fichiers massifs pouvant contenir des millions de points de mesure.

Cette information doit permettre de déterminer la taille et la vitesse des gouttes ayant traversé le faisceau malgré le bruit de mesure. Pour cela, on nous a demandé de compresser l'information contenue dans les séries temporelles mesurées par le capteur. Cette information compressée pourra ensuite être utilisée pour remonter aux caractéristiques des gouttes de pluie. Pour cela, nous allons utiliser des auto-encodeurs. L'objectif final est de trouver une méthode de restitution des gouttes robuste et non supervisée. Ici, nous nous focaliserons sur l'apprentissage des auto-encodeurs et donc sur la « compression de l'information ».

Ce projet de stage se décline en quatre sous-objectifs progressifs :

- Maîtrise des fondements théoriques - Présentation des mécanismes de base de l'apprentissage profond : architecture des réseaux de neurones, rétropropagation du gradient et algorithmes d'optimisation. (cf. Annexe A)
- Implémentation des CNN - Étudier et implémenter les réseaux convolutifs, en particulier leur extension unidimensionnelle (CNN 1D), adaptée à la détection de motifs locaux dans les signaux temporels du capteur, DBS.
- Conception d'un Auto-encodeur Convolutif (CAE) - Développer un 1D-CAE capable de compresser et reconstruire fidèlement les signatures de gouttes, en résolvant les verrous techniques liés à la normalisation et au choix des hyperparamètres.
- Étude des RNN et comparaison – Implémenter un réseau de neurones récurrent de base pour s'assurer de sa capacité à restituer les séries temporelles relativement aux performances du 1D-CAE. (Cette étape permettra de dimensionner l'architecture qui sera requise pour correctement l'intégralité de l'information portée par la série temporelle « compressée ».)

Le présent rapport est structuré en 5 chapitres. Le [Chapitre 2](#) présente le capteur ainsi que les jeux de données qu'il a permis de constituer. Le [Chapitre 3](#) détaille les principes des auto-encodeurs convolutifs (CAE) ainsi que les choix d'implémentation. Le [Chapitre 4](#) est consacré aux résultats obtenus et à leur analyse. Après une comparaison critique des architectures, le rapport se termine par une conclusion générale et des perspectives.

En [Annexe A](#), on pourra trouver les éléments concernant les fondements théoriques de l'apprentissage profond. L'[Annexe B](#) présente les réseaux convolutifs (CNN) et leurs paramètres. En [Annexe C](#), on expose les principes des réseaux récurrents (RNN) et leur application envisagée dans le projet.

1.2 Présentation du laboratoire LATMOS

Ce projet s'inscrit dans le cadre d'une collaboration scientifique de recherche universitaire avec le Laboratoire Atmosphères, Observations Spatiales (LATMOS), unité mixte de recherche (UMR 8190) rattachée à l'Université de Versailles Saint-Quentin-en-Yvelines (UVSQ), à Sorbonne Université et au Centre National de la Recherche Scientifique (CNRS).

Le LATMOS structure ses activités de recherche autour de l'étude physique et chimique des processus régissant les atmosphères terrestres et planétaires, ainsi que des interactions entre ces atmosphères et les surfaces. Les thématiques majeures du laboratoire englobent l'analyse de la couche d'ozone, l'étude de la dynamique de la moyenne et haute atmosphère, et la caractérisation instrumentale des cycles de l'eau, des nuages et des précipitations. Pour mener à bien ses objectifs, le laboratoire développe une triple compétence méthodologique s'appuyant sur la modélisation numérique, la télédétection active et passive (spatiale et terrestre), et la conception d'une instrumentation de pointe in situ.

L'équipe d'accueil au sein de laquelle ce travail a été réalisé travaille sur la microphysique des précipitations. L'un de ses enjeux majeurs est la fiabilisation et l'optimisation des outils de mesure micrométéorologiques au sol. Ces capteurs in situ fournissent les données de référence indispensables pour valider les modèles climatiques locaux et calibrer les systèmes d'observations régionaux à grande échelle, tels que le radar Doppler BASTA (Nuages et Précipitations) développé au sein du laboratoire.

L'intérêt de notre collaboration porte précisément sur le traitement avancé de séries temporelles issues d'un disdromètre optique expérimental (le modèle DBS1). L'application d'architectures d'apprentissage profond pour l'extraction automatique de caractéristiques non supervisées répond directement à un besoin d'automatisation et de traitement de données volumineuses au sein des campagnes de mesures du LATMOS

Chapitre 2 : Données et ensembles

2.1 Nature et structure des données instrumentales

Le capteur utilisé est un disdromètre optique bifaisceau (DBS) [2]. Il émet deux faisceaux laser horizontaux et parallèles, espacés verticalement de 2 mm. L'intensité lumineuse reçue par deux photodiodes définit deux signaux analogiques : la voieHaute (faisceau supérieur) et la voieBasse (faisceau inférieur). Ces signaux sont échantillonnés à 12 500 Hz.

En l'absence de précipitation, le signal est stable et maximal. Lorsqu'une goutte traverse un faisceau, elle occulte partiellement la lumière, ce qui produit une chute de tension en forme de « vallée » (Figure 2.1).

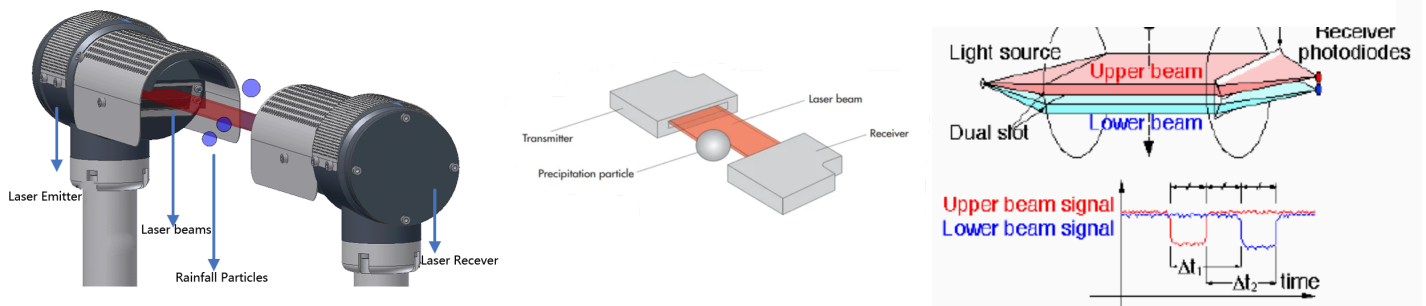


Figure 2.1 – Schéma du disdromètre DBS et illustration d'une vallée d'occultation [4].

Comme la voieBasse est située sous la voieHaute, une goutte en chute libre intersecte d'abord le faisceau supérieur puis le faisceau inférieur. Les deux vallées sont donc décalées dans le temps d'une durée Δt (Figure 2.2). Connaissant la distance entre les faisceaux (2 mm), la vitesse de chute de la goutte s'obtient par $V = 2 \text{ mm} / \Delta t$. La profondeur et la largeur de la vallée sont quant à elles liées au diamètre de la goutte.

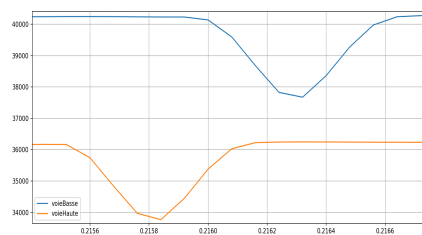


Figure 2.2 – Mise en évidence du décalage Δt entre voieHaute et voieBasse.

Variabilité morphologique des signaux selon les gouttes

La forme du signal n'est pas unique. Elle varie principalement avec le diamètre de la goutte et le nombre de gouttes traversant simultanément le faisceau. La Figure 2.3 illustre les trois cas les plus fréquents.

- **Grande goutte** : elle occulte une fraction importante du faisceau, ce qui produit une vallée profonde et large. La pente de descente et de remontée est progressive car la goutte entre et sort du faisceau de manière continue. On observe souvent un fond

de vallée arrondi en « V » ou « U ». Plus la goutte est grosse, plus la profondeur augmente et plus la durée du passage s'allonge.

- **Petite goutte** : la chute de tension est peu profonde. Lorsque la petite goutte est entièrement à l'intérieur du faisceau laser (pendant un bref instant qui dépend de sa vitesse), le signal se stabilise sur un plateau avant de remonter. La forme typique est donc : descente – plateau – remontée, (Figure 2.3b). La profondeur du plateau est directement liée au diamètre.
- **Plusieurs gouttes simultanées** : si deux gouttes ou plus traversent le faisceau en même temps, leurs vallées se superposent. Le signal peut alors présenter une descente en escalier, un seul creux plus large, ou des oscillations selon la configuration relative. Le modèle devra être capable de démêler ces contributions.

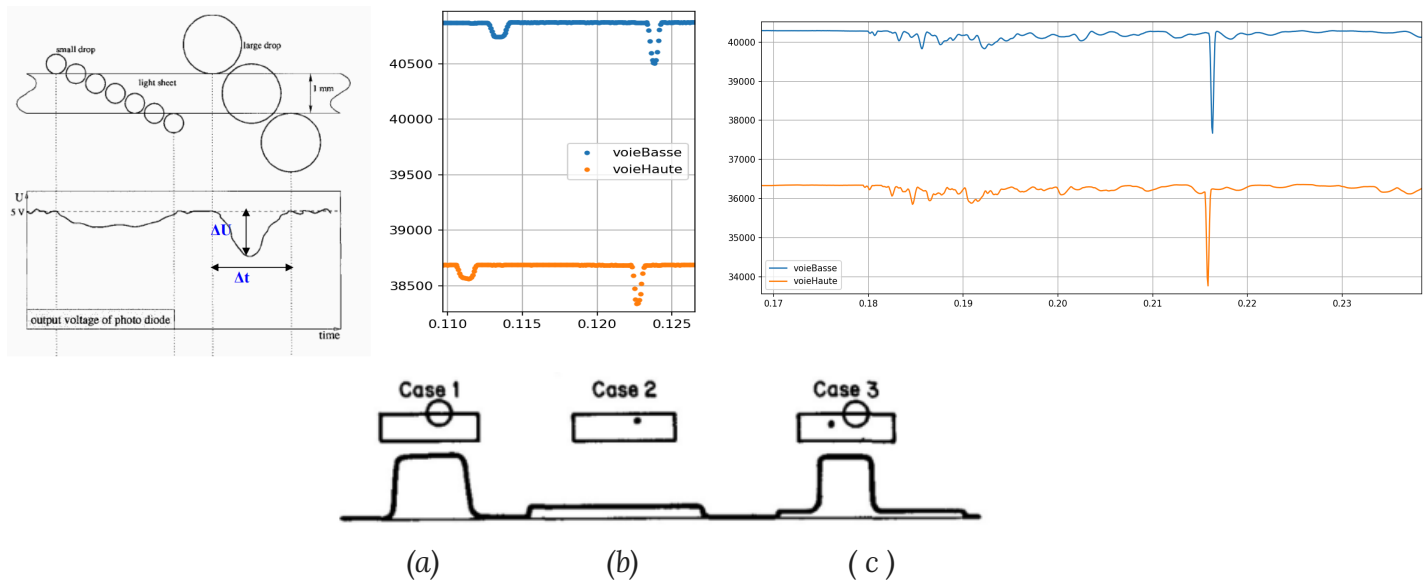


Figure 2.3 – Exemples typiques : (a) grande goutte → vallée profonde et large ;
(b) petite goutte → plateau central ; (c) deux gouttes simultanées.

Variabilité d'amplitude

En outre, les signaux bruts ne sont pas parfaitement calibrés d'un fichier à l'autre. Selon la campagne de mesure, les réglages électroniques du capteur, la température ambiante ou l'état optique des faisceaux, l'amplitude moyenne des signaux peut varier significativement. La Figure 2.4 illustre cette disparité : certains fichiers présentent une ligne de base vers 40 000 unités, d'autres vers 36 000 ou 42 000.

De plus, pour un même fichier, les amplitudes de la voieHaute et de la voieBasse ne sont pas nécessairement identiques. Cette différence provient des gains électroniques distincts ou d'un léger désalignement optique entre les deux faisceaux.

Cette variabilité externe justifie l'utilisation d'une normalisation par fenêtre (plutôt qu'une normalisation globale) lors de l'apprentissage, comme expliqué au [chapitre 4](#).

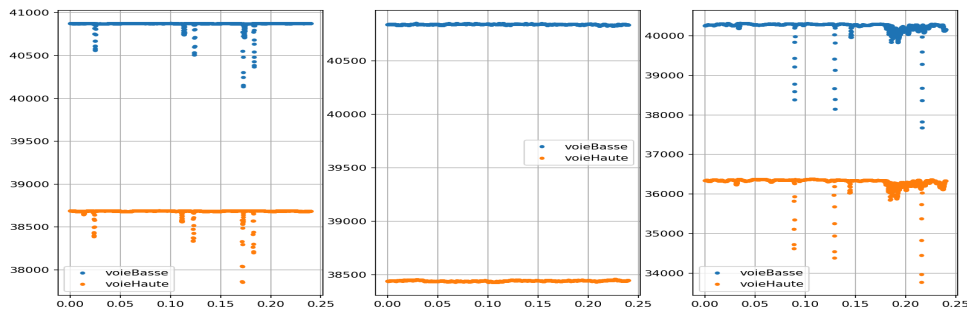


Figure 2.4 – Comparaison des amplitudes des signaux bruts pour plusieurs fichiers d'acquisition (chaque courbe correspond à un fichier différent).

Les fichiers bruts contiennent environ 11,8 millions de points par voie, soit une quinzaine de minutes d'acquisition.

2.2 Stratégie de découpage temporel (fenêtrage)

L'analyse directe de séquences aussi longues n'est pas réalisable en raison des limites de mémoire GPU et de la nécessité de fixer une taille d'entrée constante pour les couches du réseau. Nous découpons donc le signal continu en fenêtres de taille fixe, en faisant glisser une fenêtre avec un certain pas (stride).

Le choix de la taille de fenêtre doit garantir qu'une goutte ne soit jamais coupée entre deux fenêtres. D'après la loi de Gunn & Kinzer, les gouttes les plus fines (0,1 mm) chutent lentement et leur signature s'étend sur environ 70 à 80 points. Les gouttes plus grosses ont une durée d'occultation plus variable. En fixant WINDOW_SIZE = 512 points (soit environ 41 ms), on englobe une vallée complète ainsi que les plateaux de repos adjacents (Figure 2.5).

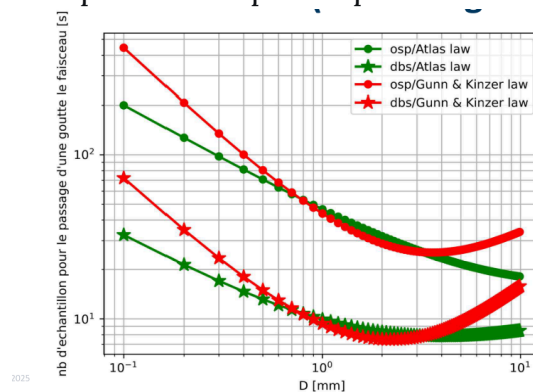


Figure 2.5 – Nombre d'échantillons mesurés lors du passage d'une particule en fonction de son diamètre.

Le pas de déplacement (stride) est fixé à 256 points, soit un recouvrement de 50 % entre fenêtres consécutives. Cela évite qu'une goutte située exactement à la frontière de deux fenêtres ne soit tronquée : elle sera capturée intégralement dans la fenêtre suivante.

2.3 Partition des ensembles d'apprentissage, validation et test

À partir d'un fichier brut, le nombre total de fenêtres exploitables est donné par :

$$N_windows = (N_points - WINDOW_SIZE) / STRIDE + 1$$

Pour un fichier de 11 814 912 points, avec `WINDOW_SIZE = 512` et `STRIDE = 256`, on obtient 46 151 fenêtres (23 076 si `STRIDE = 512`).

Ces fenêtres sont ensuite réparties aléatoirement (graine fixée à 42) selon les proportions suivantes :

- 70 % pour l'entraînement,
- 15 % pour la validation,
- 15 % pour le test.

L'ensemble de test n'est utilisé qu'à la fin pour évaluer les performances du modèle.

2.4 Pipeline d'ingestion logicielle (Dataset et DataLoader PyTorch)

Pour charger efficacement les données sur GPU, nous avons développé une classe `PrecipDataset` héritant de `torch.utils.data.Dataset`. Elle lit les fichiers binaires via la fonction `dbs_load_bin` (fournie par le laboratoire), applique le découpage en fenêtres, et retourne les signaux sous forme de tenseurs.

Les échantillons sont encapsulés dans un `DataLoader` avec un `batch_size = 32`. L'ensemble d'entraînement est mélangé (`shuffle=True`) à chaque époque pour éviter l'apprentissage de corrélations temporelles parasites.

Chapitre 3 : Auto-encodeur convolutif 1D (CAE)

3.1 Principe des auto-encodeurs

Un auto-encodeur (AE) est un réseau de neurones entraîné à recopier son entrée en sortie, mais en passant par une représentation intermédiaire de plus faible dimension, appelée espace latent. Cette contrainte force le modèle à ne conserver que les variations essentielles des données et à éliminer le bruit. L'apprentissage est non supervisé : on ne fournit aucune étiquette, seulement les signaux bruts.

Dans notre projet, l'entrée est une fenêtre temporelle de 512 points (une voie) ou la concaténation des deux voies (1024 points). Le modèle doit apprendre la morphologie typique d'une vallée de goutte sans annotation manuelle.

Architecture encodeur-décodeur

Un auto-encodeur se compose de deux parties symétriques (Figure 3.1) :

- Encodeur : projette l'entrée x vers un vecteur latent $z = f_{\theta}(x)$ de dimension réduite.
- Décodeur : reconstruit $\hat{x} = g_{\phi}(z)$ à partir de z

La qualité de la reconstruction est mesurée par une fonction de perte. Nous utilisons l'erreur quadratique moyenne (MSE) :

$$L = \frac{1}{N} \sum_{i=1}^N \left\| x_i - \hat{x} \right\|^2$$

L'espace latent joue le rôle d'un goulot d'étranglement informationnel. En réduisant sa dimension, on espère que chaque variable latente code une caractéristique physique indépendante (diamètre, vitesse, etc.).

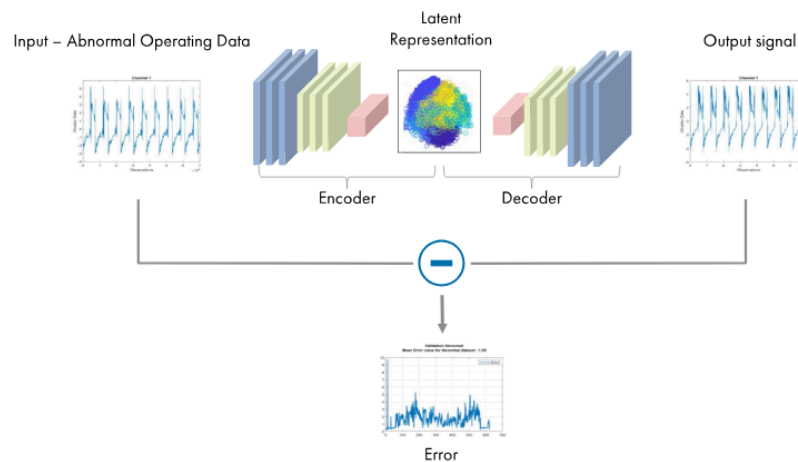


Figure 3.1 – Schéma général d'un auto-encodeur avec un espace latent

Adaptation aux séries temporelles : choix du CNN 1D

Les séries temporelles issues du capteur présentent des motifs locaux (descente rapide, plateau, remontée). Un réseau convolutif 1D (CNN 1D) est naturellement adapté à leur détection grâce à ses filtres glissants et au partage des poids. Les principes généraux des CNN sont détaillés en [Annexe B](#). L'auto-encodeur développé ici est un auto-encodeur convolutif 1D (1D-CAE), où l'encodeur utilise des couches Conv1D et le décodeur des couches ConvTranspose1D.

3.4 Implémentation du CAE 1D

3.4.1 Première version : une seule voie (voieBasse)

Tous les essais ont débuté avec la voieBasse seule, afin de valider l'architecture avant d'intégrer la seconde voie.

Paramètres de convolution

Pour chaque couche Conv1D de l'encodeur, les hyperparamètres suivants sont fixés :

- `kernel_size = 4`
- `stride = 2`
- `padding = 1`

Avec ces valeurs, la longueur temporelle est divisée par deux à chaque étape (calcul détaillé en [Annexe B.2.3](#)). Trois blocs convolutionnels successifs sont appliqués, réduisant une entrée de 512 points à 64 points dans l'espace des caractéristiques.

Étape	Type	Dimensions d'entrée → sortie
Entrée		(batch, 1, 512)
Bloc 1	Conv1D(1→16, k=4, s=2, p=1) + BatchNorm + LeakyReLU(0.5)	(batch, 16, 256)
Bloc 2	Conv1D(16→32, k=4, s=2, p=1) + BatchNorm + LeakyReLU(0.5)	(batch, 32, 128)
Bloc 3	Conv1D(32→64, k=4, s=2, p=1) + BatchNorm + LeakyReLU(0.5)	(batch, 64, 64)
Flatten		(batch, 4096)
Dense	Linear(4096 → 32)	vecteur latent z de dimension 32

Espace latent : la dimension 32 a été choisie après plusieurs essais (configuration 2, chapitre 4). Une dimension plus petite (8) entraîne une perte d'information, une dimension plus grande (64) n'apporte pas de gain significatif.

Architecture du décodeur (symétrique)

- Couche dense : Linear(32 $\rightarrow$ 4096), puis reshape en (batch, 64, 64).
- Trois couches ConvTranspose1D (k=4, s=2, p=1) :
 - 64 $\rightarrow$ 32 canaux, sortie (batch, 32, 128)
 - 32 $\rightarrow$ 16 canaux, sortie (batch, 16, 256)
 - 16 $\rightarrow$ 1 canal, sortie (batch, 1, 512)

Chaque couche de convolution transposée double la longueur temporelle. La dernière couche ne comporte ni BatchNorm ni activation (signal reconstruit brut).

Normalisation par lot et fonction d'activation

Chaque couche de convolution (et de convolution transposée, sauf la dernière) est suivie d'une BatchNorm1d pour stabiliser l'apprentissage. La fonction d'activation choisie est LeakyReLU(0.5), qui évite le problème des neurones morts rencontré avec la ReLU classique.

Optimisation

L'optimiseur Adam est utilisé avec un taux d'apprentissage initial de 10^{-3}

. La fonction de perte est la MSE calculée entre le signal normalisé en entrée et sa reconstruction.

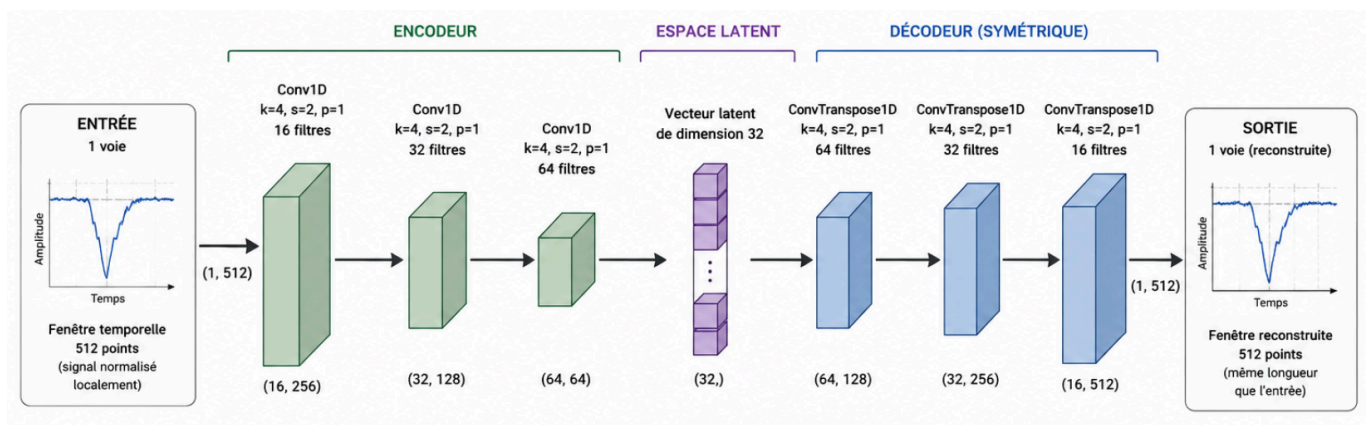


Figure 3.2 – Architecture en sable détaillée du 1D-CAE illustrant les dimensions des tenseurs et l'évolution des cartes de caractéristiques

3.4.2 Extension à deux voies

Après validation du modèle sur une seule voie, l'architecture a été étendue pour traiter simultanément les deux signaux (voieBasse et voieHaute). L'entrée du réseau est alors un tenseur de forme (batch, 2, 512) : les deux voies sont placées côte à côte comme deux canaux distincts, à l'image des trois canaux (rouge, vert, bleu) d'une image couleur. Cette représentation conserve l'indépendance temporelle de chaque voie tout en permettant au réseau d'apprendre leurs corrélations.

La première couche Conv1D est modifiée pour accepter 2 canaux d'entrée (in_channels=2). Le reste de l'architecture est identique à celle de la section [3.4.1](#), à l'exception de la dimension de l'espace latent, portée à 64 (contre 32 pour une seule voie) pour offrir une capacité suffisante au codage conjoint des deux signaux.

Une autre configuration a également été testée : la concaténation linéaire des deux voies bout à bout en un seul vecteur de 1024 points (entrée (batch, 1, 1024)). Cette approche s'est révélée moins performante en termes de fidélité de reconstruction. Les résultats comparatifs sont détaillés au chapitre 4 ([section 4.3](#)).

Chapitre 4 : Analyse des résultats

4.1 Démarche expérimentale

Plusieurs configurations du CAE 1D ont été testées de manière incrémentale. L'objectif est d'évaluer l'impact des hyperparamètres (nombre de couches convolutives, dimension de l'espace latent, type de normalisation) sur la qualité de reconstruction des signaux de précipitations. Tous les essais préliminaires ont été réalisés sur la voie Basse seule, avant d'étendre le modèle aux deux voies. Les métriques principales sont la MSE et le coefficient de détermination R^2 sur l'ensemble de test.

Le coefficient de détermination R^2 mesure la part de variance expliquée par le modèle. Il est défini par :
$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$
 où y_i est le signal original, $\hat{y}_i$ la reconstruction et $\bar{y}$ la moyenne du signal original. Une valeur proche de 1 indique une bonne fidélité de reconstruction [4].

4.2 Configurations

4.2.1 : modèle à 2 convolutions (sous-apprentissage)

Premier essai avec seulement deux couches convolutives dans l'encodeur (et symétriquement deux couches ConvTranspose). L'espace latent est fixé à 16, le stride de fenêtrage à 512 (pas de recouvrement). Le nombre total de paramètres est de 40 209. L'entraînement dure 40 époques.

Les pertes stagnent rapidement :

```
Epoch 1 : train_loss = 0,08068, val_loss = 0,03763
Epoch 20 : train_loss = 0,02972, val_loss = 0,03005
Epoch 40 : train_loss = 0,02841, val_loss = 0,02834
```

L'examen visuel (Figure 4.1) montre que le modèle lisse excessivement les vallées et produit parfois des reconstructions aberrantes (lignes droites moyennes). La capacité du modèle est insuffisante pour capturer la morphologie fine des gouttes.

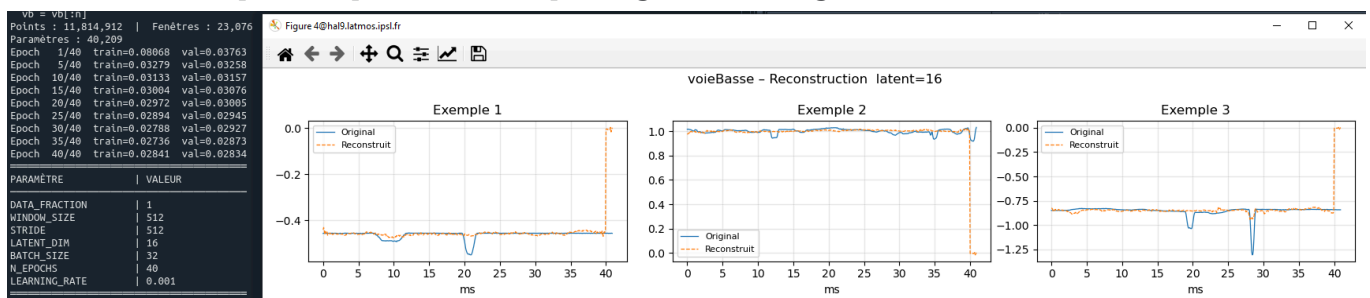
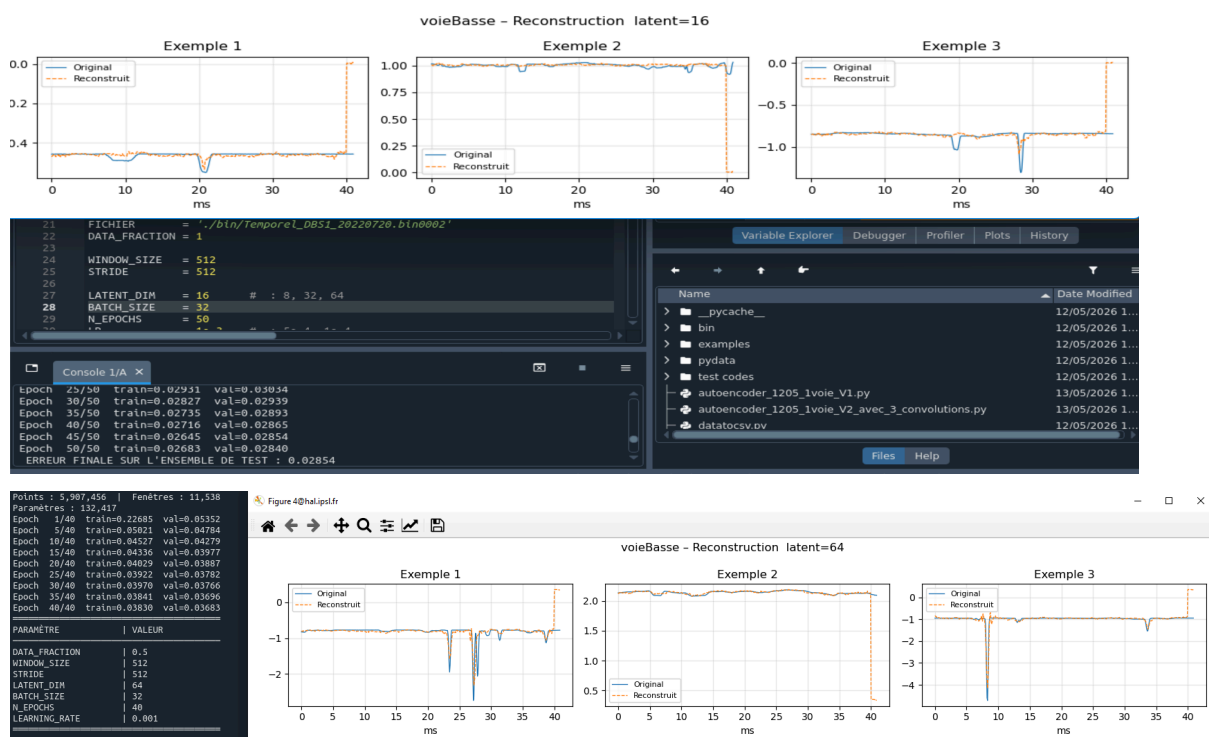


Figure 4.1 – Exemples de reconstruction pour la configuration 1 (modèle à 2 convolutions).

4.2.2 Configuration 2 : trois convolutions et choix de la dimension latente

Le nombre de couches convolutives est porté à trois (encodeur et décodeur). Les hyperparamètres de convolution sont conservés ($k=4$, $s=2$, $p=1$). Le nombre de paramètres atteint 1 082 241. Pour accélérer les tests, seul 50 % des données est utilisé (11 538 fenêtres) sur 40 époques. Deux dimensions de l'espace latent sont comparées :

- Latent = 8 : la perte finale reste élevée ($\text{val_loss} = 0,04852$). La compression trop forte fait perdre la signature géométrique des gouttes.
- Latent = 64 : la convergence s'améliore ($\text{val_loss} = 0,03683$). Cependant, les résultats sont quasiment identiques à ceux obtenus avec latent = 32 lors d'essais complémentaires (non montrés). Une dimension de 32 suffit donc.



La Figure 4.2 compare visuellement les reconstructions pour latent=8 et latent=64.

4.2.3 Configuration 3 : normalisation globale et problème d'offset

Le modèle à trois couches est entraîné sur 100 % des données (46 151 fenêtres, $\text{STRIDE}=256$) pendant 70 époques. Les entrées sont normalisées de façon globale : soustraction de la moyenne générale et division par l'écart-type global de la série brute. Les métriques finales sont excellentes :

- MSE test = 0,00489
- R^2 test = 99,52 %

L'évolution des pertes est satisfaisante (Figure 4.3a). La projection ACP de l'espace latent (Figure 4.3b) montre que les deux premières composantes capturent **90 %** de la variance (PC1 = 74,65 %, PC2 = 15,41 %).

Cependant, l'examen détaillé des reconstructions (Figure 4.3c) révèle **un décalage vertical** systématique (offset) entre le signal original et sa reconstruction. Bien que la forme en vallée soit respectée, la ligne de base reconstruite est translatée vers le haut ou vers le bas. Cet offset provient des dérives lentes de l'intensité du laser que la normalisation globale ne peut corriger localement. Or, la profondeur des vallées (liée au diamètre) nécessite une reconstruction sans biais vertical. Cette configuration est donc inutilisable en pratique.

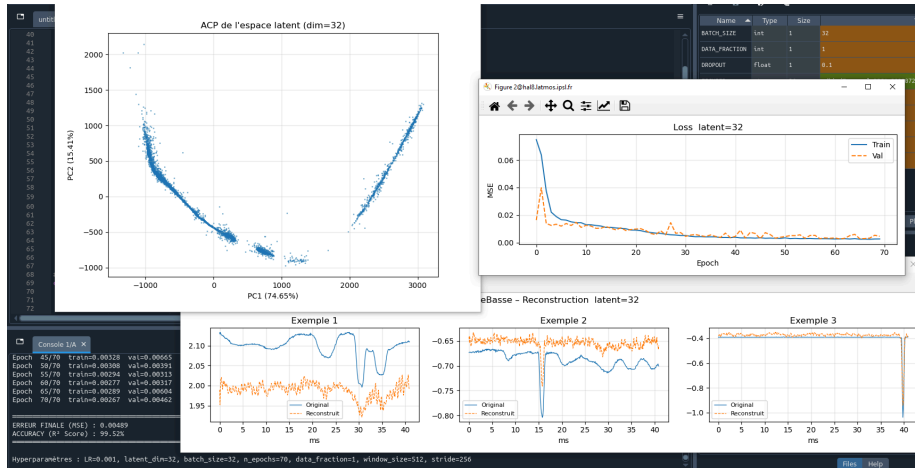


Figure 4.3 – Configuration 3 (normalisation globale) : (a) courbes de perte, (b) projection ACP de l'espace latent, (c) exemple de reconstruction montrant l'offset vertical.

4.2.4 Configuration 4 : normalisation locale (z-score par fenêtre)

Pour éliminer l'offset, le prétraitement est modifié : chaque fenêtre de 512 points est normalisée indépendamment par son propre z-score : $Window_{norm} = \frac{Window - \mu_{Window}}{\delta_{Window} + \varepsilon}$

Ainsi, le réseau ne voit plus que la forme relative de la vallée, indépendamment du niveau absolu du faisceau. La même architecture (trois couches, latent=32) est réentraînée.

Les résultats visuels (Figure 4.4) montrent un alignement parfait entre l'entrée et la reconstruction, sans aucun décalage vertical. En revanche, les métriques globales se dégradent : MSE test = 0,1419, $R^2 = 85,8 \%$.

La projection ACP de l'espace latent (Figure 4.4b) fait apparaître un nuage dispersé (PC1+PC2 = 15 % de variance). Cette dégradation s'explique par l'amplification du bruit électronique dans les zones sans pluie (division par un écart-type local très faible). Toutefois, la fidélité géométrique des vallées étant primordiale pour l'extraction physique ultérieure (diamètre, vitesse), la normalisation locale est retenue.

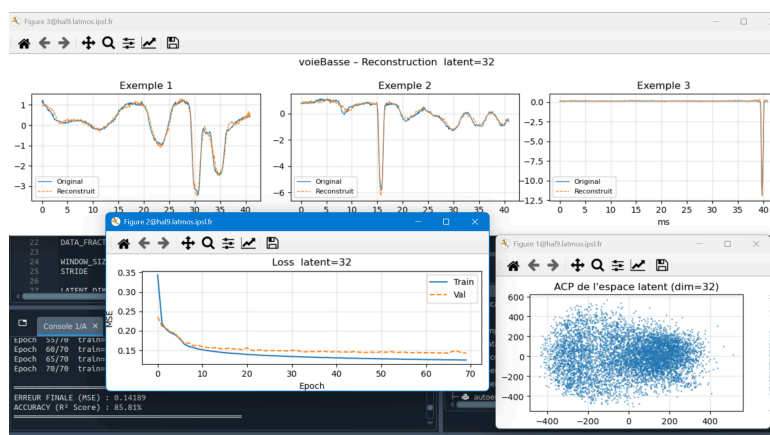


Figure 4.4 – Configuration 4 (normalisation locale) : (a) courbes de perte, (b) projection ACP, (c) exemples de reconstruction parfaitement alignés.

Le Tableau 4.1 résume la comparaison entre les deux normalisations.

Critère	Globale	Locale
MSE_{test}	0,00489	0,1419
R^2_{test}	99,5 %	85,8 %
Variance ACP (PC1+PC2)	90 %	15 %
Rendu géométrique	offset vertical	alignement parfait
Utilité pour l'extraction physique	non	oui

Tableau 4.1 – Comparaison normalisation globale vs locale

4.3 Extension à deux voies

Après validation sur une voie, le modèle est adapté aux deux signaux (voieBasse et voieHaute). Deux architectures ont été comparées :

1. Entrée à deux canaux : tenseur (batch, 2, 512). La première couche Conv1D utilise in_channels=2. Espace latent = 64.
2. Concaténation linéaire : les deux voies sont mises bout à bout en un vecteur de 1024 points, entrée (batch, 1, 1024). Même architecture mais latent = 64.

La configuration à deux canaux donne une reconstruction nettement meilleure (Figure 4.5). Les deux voies sont reconstruites simultanément avec une faible erreur. La concaténation linéaire dégrade la qualité, probablement car elle force le réseau à apprendre une

corrélation temporelle artificielle entre la fin de la voieBasse et le début de la voieHaute. Seule l'architecture à deux canaux est conservée pour la suite.

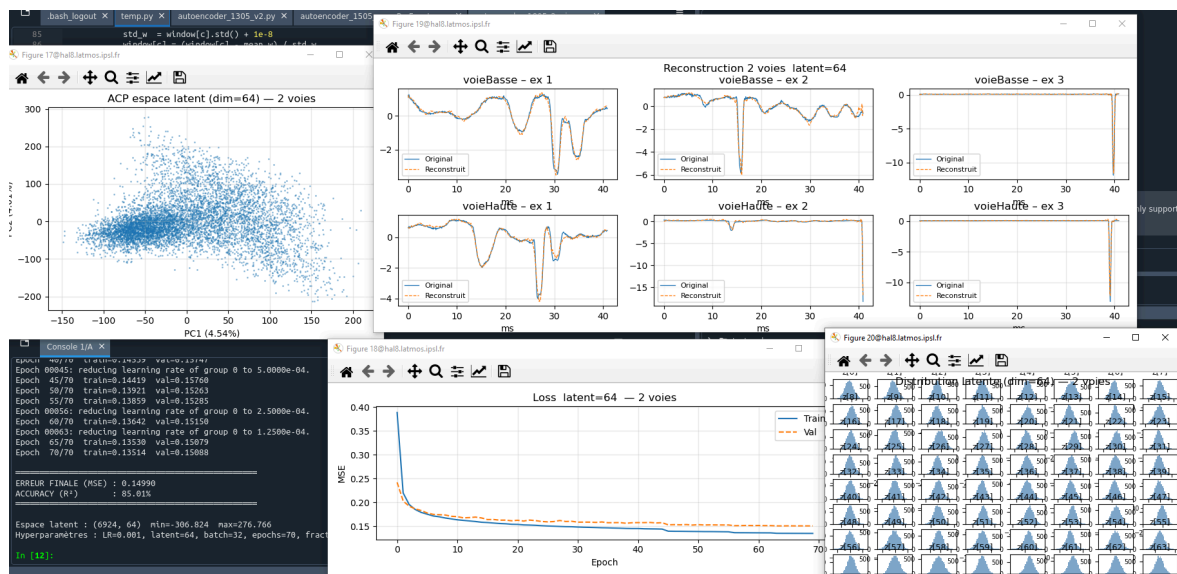


Figure 4.5 – Reconstruction des deux voies (entrée à deux canaux) .

4.4 Analyse de l'indépendance des variables latentes

L'espace latent du modèle final (deux canaux, dimension 64) est analysé à l'aide de la matrice de corrélation linéaire (Figure 4.6). Sur la diagonale, chaque variable est naturellement corrélée à elle même (coefficient égal à 1). En dehors de la diagonale, l'ensemble des coefficients de corrélation entre variables distinctes est très proche de zéro. La Figure 4.7 montre les histogrammes des distributions (diagonale du pair plot) : elles suivent une allure gaussienne centrée sur 0. Les nuages de points (sous la diagonale) sont ronds et diffus, confirmant l'absence de relation linéaire.

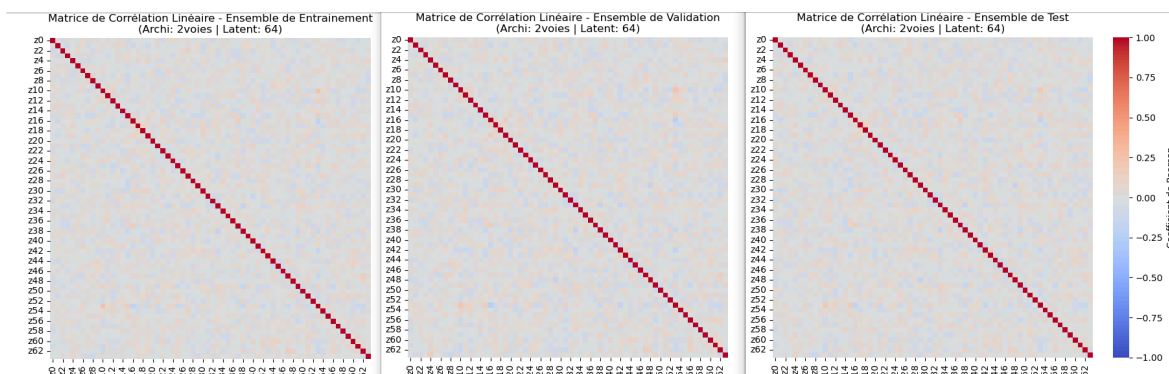


Figure 4.6 – Matrice de corrélation (heatmap) des 64 variables latentes (ensemble de test). Les valeurs sont proches de 0 hors diagonale.

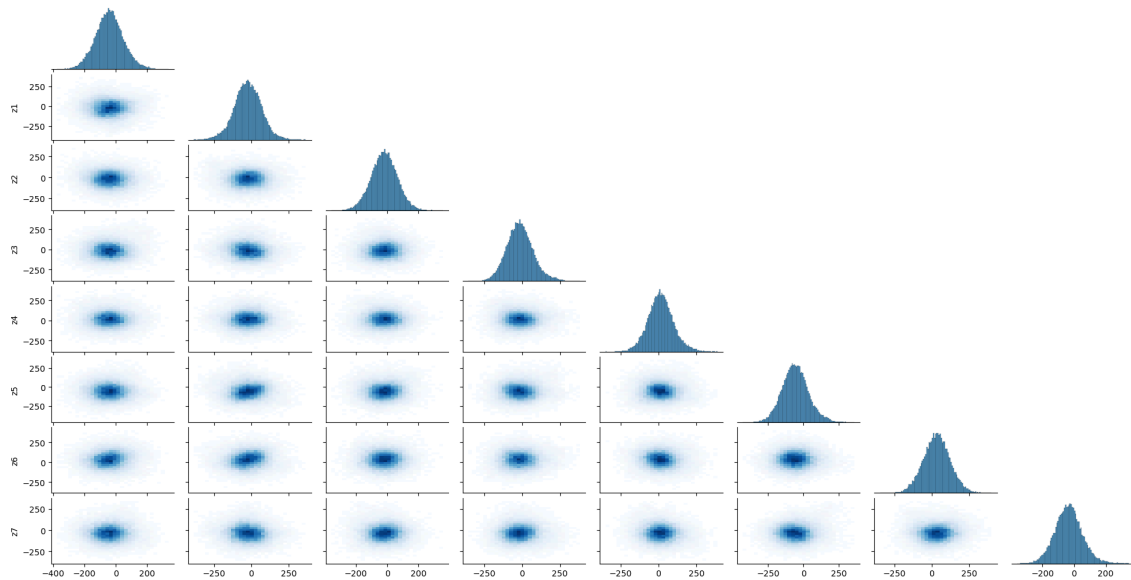


Figure 4.7 – Distributions (histogrammes) et nuages de points (pair plot) pour les premières dimensions latentes (exemple).

Ces observations indiquent que les variables latentes sont linéairement décorrélées, donc non redondantes. Chaque dimension semble coder une caractéristique distincte du signal (potentiellement le diamètre, la vitesse, la forme fine, etc.). Une perspective future consistera à identifier le rôle physique de chacune, par exemple en modulant une variable latente à la fois et en observant l'effet sur la reconstruction.

4.5 Bilan des résultats

- Un modèle à trois couches convolutives est nécessaire (deux couches sont insuffisantes).
- La dimension de l'espace latent 32 (une voie) ou 64 (deux voies) est un bon compromis.
- La normalisation locale par fenêtre est indispensable pour supprimer l'offset vertical et obtenir une reconstruction géométrique fidèle, même si les métriques globales sont moins bonnes.
- L'entrée à deux canaux (voieBasse et voieHaute) donne de meilleurs résultats qu'une simple concaténation.
- Les variables latentes sont décorrélées, ce qui suggère un codage non redondant des caractéristiques physiques des gouttes.

Chapitre 5 : Conclusion et perspectives

5.1 Bilan du projet

Ce stage visait à compresser l'information contenue dans les séries temporelles d'un disdromètre optique (DBS) à l'aide d'auto-encodeurs convolutifs 1D, afin de faciliter l'extraction ultérieure des caractéristiques physiques des gouttes de pluie (diamètre, vitesse). Les principaux résultats obtenus sont les suivants :

- Un 1D-CAE à trois couches convolutives (encodeur) et trois couches transposées (décodeur) s'avère nécessaire pour capturer la morphologie fine des vallées. Une architecture à deux couches conduit à un sous-apprentissage marqué.
- La normalisation locale par fenêtre (z-score calculé sur chaque fenêtre indépendamment) est indispensable pour éliminer le décalage vertical (offset) induit par les dérives lentes du laser. En dépit d'une augmentation de la MSE et d'une baisse du R2, cette stratégie garantit une reconstruction géométriquement fidèle, condition préalable à toute estimation physique.
- L'extension aux deux voies (voieBasse et voieHaute) est réalisée efficacement en utilisant deux canaux d'entrée (tenseur (batch, 2, 512)). Cette approche donne de meilleurs résultats que la simple concaténation linéaire des deux signaux.
- L'espace latent du modèle final (64 dimensions pour deux voies) présente des variables indépendantes (matrice de corrélation quasi diagonale), ce qui suggère un codage non redondant des caractéristiques des gouttes.

5.2 Limites et difficultés rencontrées

- Les métriques globales (MSE, R2) se sont révélées trompeuses : une excellente valeur peut cacher un offset vertical rédhibitoire. L'évaluation visuelle reste indispensable.
- La normalisation locale amplifie le bruit électronique dans les fenêtres sans pluie, ce qui disperse l'espace latent et dégrade les scores numériques. Toutefois, ce défaut n'affecte pas la reconstruction des vallées.
- L'absence de vérité terrain (labels des diamètres et vitesses) empêche une validation directe de la qualité physique de l'espace latent. Les analyses de corrélation ne sont qu'indirectes.

5.3 Perspectives

Pour l'extraction physique : les paramètres de normalisation (moyenne de chaque fenêtre) doivent être sauvegardés. En effet, le réseau ne travaille que sur des signaux centrés-réduits. Pour retrouver le signal original (et donc les amplitudes absolues des vallées), il faut appliquer la transformation inverse :

$$x_{original} = x_{norm} * \sigma_{fen} + \mu_{fen}$$

Ainsi, pour une fenêtre donnée, l'information utile se résume à :

- une voie : 32 variables latentes + 1 moyenne + 1 écart-type (soit 34 valeurs au lieu de 512 points bruts) ;
- deux voies : 64 variables latentes + 2 moyennes + 2 écarts-type (soit 68 valeurs au lieu de 1024 points).

Cette compression (facteur ~ 15) ouvre la voie à un stockage efficace et à une analyse ultérieure (clustering, régression supervisée) pour prédire diamètre et vitesse.

Travaux futurs :

- Utiliser l'espace latent comme entrée d'un réseau récurrent (LSTM, voir [Annexe C](#)) pour modéliser la dynamique temporelle des précipitations.
- Mettre en place une procédure supervisée (si des labels deviennent disponibles) pour apprendre directement la correspondance entre l'espace latent et les paramètres (D, V, t, N) .
- Explorer des variantes (auto-encodeurs variationnels, convolutions dilatées) pour améliorer la robustesse au bruit.

Bibliographie

- [1] Armetta, A. (2006) Etude bibliographique des appareils de mesures de précipitations installés au Col de Porte. Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*.
- [2] Delahaye, J. Y., Barthès, L., Golé, P., Lavergnat, J., & Vinson, J. P. (2006). A dual-beam spectropluviometer concept. *Journal of Hydrology*, 328(1-2), 110-120.
- [3] X. Zhang, X. Zhou, M. Lin and J. Sun, "ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices," *arXiv preprint arXiv:1803.01271*, 2018. [Online].
- [4] Nguyen Van, L., Le, X.-H., Nguyen, G. V., Yeon, M., Do, M.-T. T., & Lee, G. (2023). Evaluation of Numerous Kinetic Energy-Rainfall Intensity Equations Using Disdrometer Data. *Remote Sensing*, 15(1), 156 .
- [5] Lasse Schmidt (2021), "Time Series Classification with Convolutions", medium
- [6] Saura, N. (2021). *Modélisation du tenseur de contraintes sous-maîles par réseau de neurones à convolutions 3D en Turbulence homogène isotrope*. Thèse de doctorat, Université de Lille. (NNT : 2021LILUI046), (tel-03482852).

Annexes

Annexe A : Rappels sur l'apprentissage profond

A.1 Architecture d'un réseau de neurones

Un réseau de neurones artificiel est un graphe orienté de couches de neurones. Chaque neurone calcule une combinaison linéaire de ses entrées, à laquelle est appliquée une fonction d'activation non-linéaire. C'est cette non-linéarité qui confère au réseau sa capacité d'approximation universelle. La sortie d'un neurone j dans la couche h s'écrit :

$$a_j^{(h)} = f \left(\sum_i w_{ji}^{(h)} x_i + b_j^{(h)} \right) \quad (2.1)$$

où w_{ji} est le poids de connexion, b_j le biais,

et f la fonction d'activation.

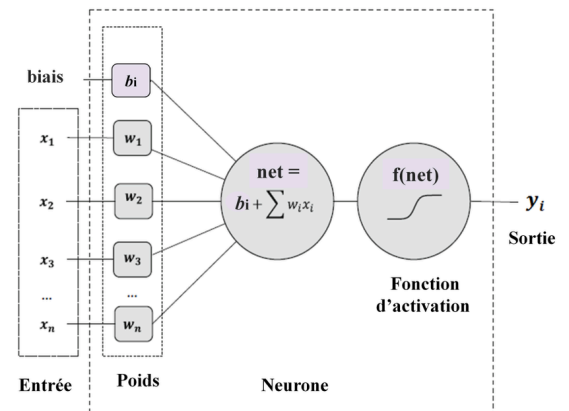
Les fonctions d'activation les plus courantes sont :

- $ReLU(x) = \max(0, x)$ (2.2a)

- $sigmoid(x) = \frac{1}{(1 + \exp(-x))}$ (2.2b)

- $tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$ (2.2c)

- $LeakyReLU(x) = \max(\alpha x, x)$ (2.2d)



La ReLU est aujourd'hui dominante dans les couches cachées. Elle évite le problème de **disparition du gradient** qui affecte le sigmoid et la tanh pour les réseaux profonds, et sa simplicité de calcul

accélère l'entraînement. Cependant, la **Leaky ReLU** intervient précisément pour corriger le principal défaut de la ReLU classique : **le problème des neurones morts (Dying ReLU problem)**.

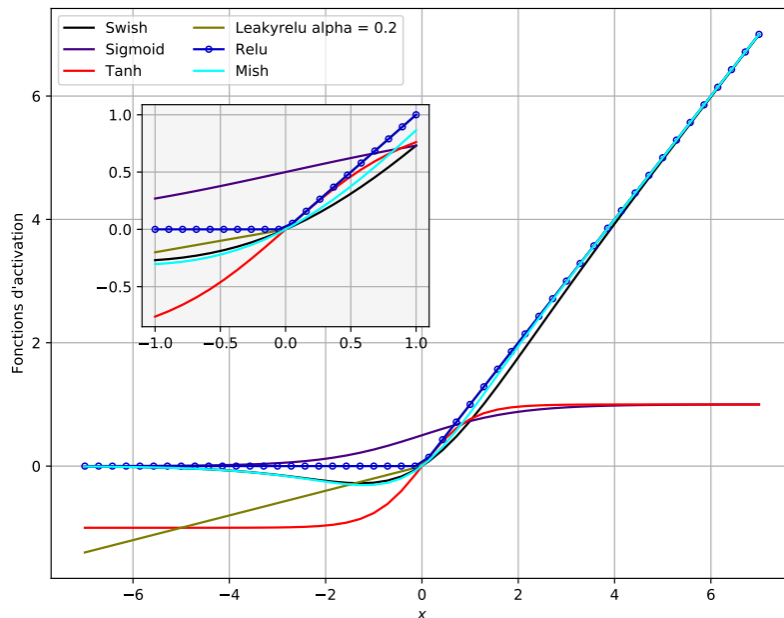


Figure 2.1 Graphiques de différentes fonctions d'activation: ReLU, Leakyrelu, Sigmoid et Tanh [6]

L'architecture classique d'un réseau totalement connecté (MLP) enchaîne plusieurs couches fully-connected suivies chacune d'une activation. La dernière couche produit la sortie — une valeur réelle pour la régression, ou un vecteur de probabilités (via Softmax) pour la classification.

- **La couche d'entrée** : qui reçoit les données brutes.
- **Les couches cachées** : où s'effectuent les transformations non-linéaires et l'apprentissage des caractéristiques. Chaque neurone d'une couche est connecté aux neurones de la couche précédente par des poids (w) et possède un biais (b).
- **La couche de sortie** : qui fournit la prédiction finale ou la classification.

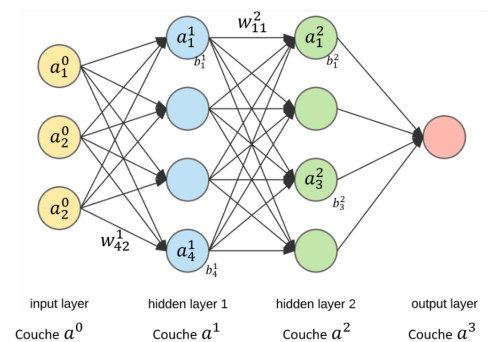


Figure 2.2 "Architecture d'un MLP : couches d'entrée, cachées et de sortie"

A.2 Fonction de perte et optimisation

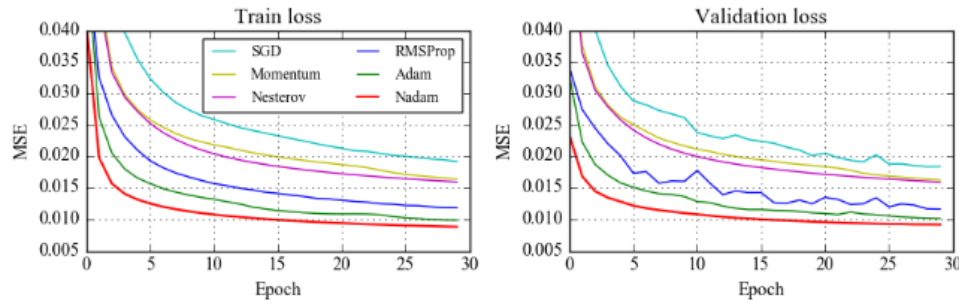
La performance d'un modèle est mesurée par une **fonction de perte** (Loss Function), qui calcule l'erreur entre la prédiction du réseau ($\hat{y}$) et la valeur réelle cible (y).

- **Entropie croisée (Cross-Entropy)** : couramment utilisée pour la classification.
- **Erreur quadratique moyenne (MSE)** : privilégiée pour les problèmes de régression.

$$L = MSE = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 \quad (2.3)$$

L'optimisation consiste à minimiser cette erreur en ajustant de manière répétée les paramètres du réseau via un **optimiseur**. L'algorithme d'optimisation le plus courant est la

descente de gradient (**SGD**), dont des versions avancées comme **Adam** adaptent le taux



d'apprentissage pour chaque paramètre, améliorant ainsi la vitesse de convergence.

Figure 2.3 : Comparaison des courbes de perte : SGD vs Momentum vs Adam – convergence

A.3 Rétropropagation du gradient

La rétropropagation (*Backpropagation*) est le mécanisme fondamental permettant de mettre à jour les poids et biais en fonction de l'erreur calculée en sortie. Elle consiste à calculer les dérivées partielles de la fonction d'erreur par rapport à chaque paramètre en partant **de la dernière couche vers la première**.

L'ajustement du poids w s'effectue selon la règle :

$$w_t = w_{t-1} - \eta \cdot \frac{\partial L}{\partial w}$$

où η représente le taux d'apprentissage (*learning rate*).

Pour un poids $w_{jk}^{(l)}$, la contribution à l'erreur est :

$$\frac{\partial L}{\partial w_{jk}^{(l)}} = \frac{\partial L}{\partial a_j^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{jk}^{(l)}}$$

Ce processus itératif, partant de la couche de sortie vers la couche d'entrée, assure que chaque paramètre est ajusté proportionnellement à sa responsabilité dans l'erreur de prédiction.

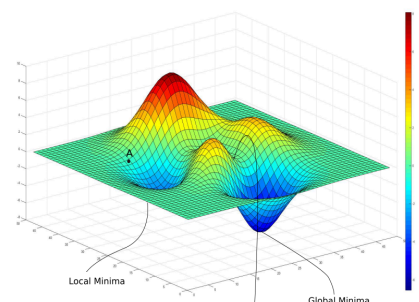


Figure 2.4 : Visualisation de l'espace des poids montrant le minimum global et un minimum local.

Annexe B : Réseaux de neurones convolutifs (CNN)

B.1 Principe général

Les réseaux convolutifs (CNN) sont conçus pour traiter des données structurées en grille (images, séries temporelles). Leur particularité est d'utiliser l'opération de convolution au lieu de la multiplication matricielle dense des perceptrons multicouches (MLP). Cette approche repose sur trois idées clés :

- Connectivité locale : chaque neurone ne se connecte qu'à une petite région de l'entrée (champ récepteur).
- Partage des paramètres : le même filtre (noyau) est appliqué à toutes les positions.
- Invariance par translation : un motif appris (bord, creux, pic) est détectable n'importe où dans le signal.

Ces propriétés réduisent drastiquement le nombre de paramètres et facilitent la généralisation.

B.2 Convolution 2D (images) et 1D (séries temporelles)

B.2.1 Définition mathématique

Pour un signal d'entrée 1D noté x de longueur L et un filtre w de taille K (noyau), la sortie y à la position i est :

$$y_k = \sum_{i=0}^{K-1} x_{k \cdot S + i} \cdot w_i + b \quad (\text{B.2.1})$$

Où :

- x est le signal d'entrée.
- w représente les poids du noyau de taille K .
- S est le **stride** (le pas de déplacement).
- b est le biais appris.

En 2D, la formule s'étend à deux dimensions (hauteur et largeur).

$$S(i, j) = \sum_{m=0}^{n-1} \sum_{k=0}^{n-1} X(i + m, j + k) \cdot K(m, k) + b \quad (\text{B.2.2})$$

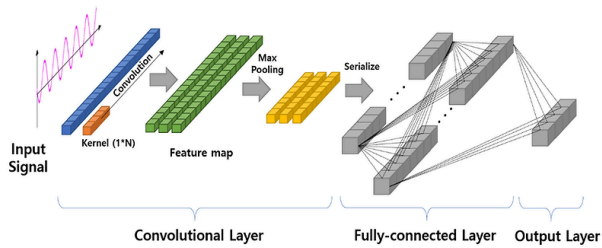


Figure B.1.1 – Illustration d'une convolution 1D.

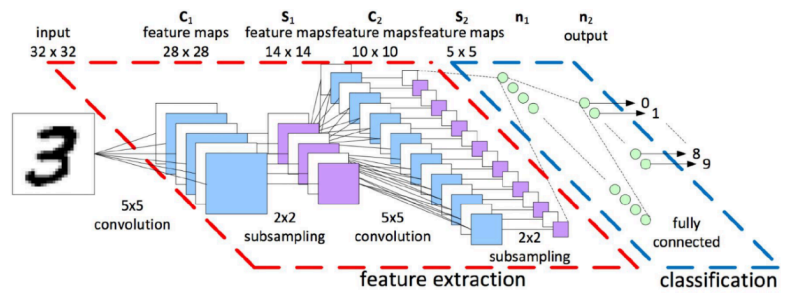


Figure B.1.2 – Illustration d'une convolution 2D.

B.2.2 Paramètres d'une couche convolutionnelle

Paramètre	Rôle
Kernel size (K)	Taille du filtre (nombre de points consécutifs couverts)
Stride (S)	Pas de déplacement entre deux applications du filtre. S=1 → application à chaque point ; S=2 → sous-échantillonnage.
Padding (P)	Ajout de zéros aux bords pour contrôler la dimension de sortie.

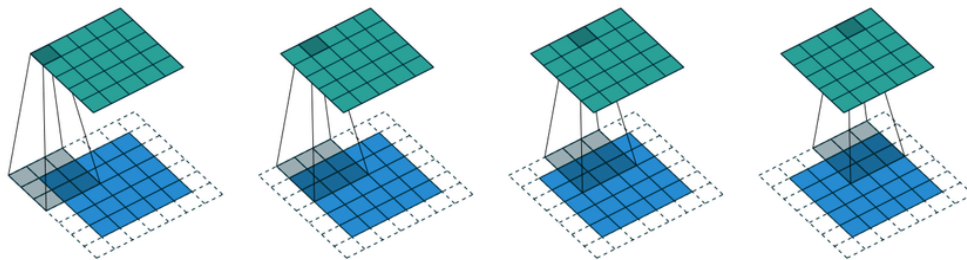


Figure B.2 – Animation de l'opération de convolution 2D avec stride et padding

B.2.3 Calcul de la dimension de sortie

Pour une entrée de longueur L_{in} , un filtre de taille K , un stride S et un padding P , la longueur de sortie L_{out} est :

$$L_{out} = \left(\frac{L_{in} + 2 \times padding - kernel_size}{stride} \right) + 1 \quad \text{issue de [5]}$$

Exemple 1: **kernel_size** = 4, **stride** = 2, et **padding** = 1 :

$$L_{out} = \left(\frac{L_{in} + 2 \times 1 - 4}{2} \right) + 1 = \left(\frac{L_{in} - 2}{2} \right) + 1 = \frac{L_{in}}{2} - 1 + 1 = \frac{L_{in}}{2}$$

En 2D, on applique la même formule à la hauteur et à la largeur.

pour la **convolution transposée** : La longueur temporelle en sortie est régie par la formule d'expansion arithmétique suivante :

$$L_{out} = (L_{in} - 1) \times stride - 2 \times padding + kernel_size \quad \text{issue de [5]}$$

Exemple 2: (k=4, s=2, p=1)

$$L_{out} = (L_{in} - 1) \times 2 - 2 \times 1 + 4 = 2L_{in} - 2 - 2 + 4 = 2L_{in}$$

B.3 Architecture classique d'un CNN

Une chaîne de traitement typique alterne :

1. Couche convolutionnelle : extraction de motifs locaux.
2. Fonction d'activation (souvent ReLU) : non-linéarité.
3. Couche de pooling (optionnelle) : réduction de dimension (max-pooling ou average-pooling).

Le max-pooling remplace une fenêtre de taille p par sa valeur maximale, introduisant une invariance aux petites translations.

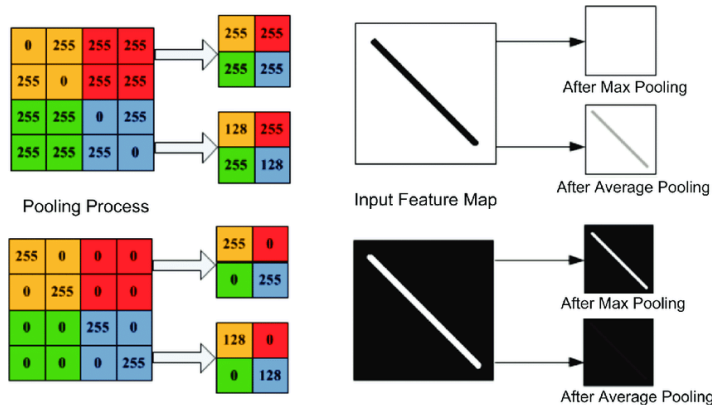


Figure B.3 : Comparaison entre Max-Pooling et Average-Pooling sur une feature map-2D

En fin de chaîne, les cartes de caractéristiques sont aplaties (flatten) puis traitées par une ou plusieurs couches fully connected (denses) pour produire la sortie (classification, régression, etc.).

B.4 Pourquoi les CNN 1D sont adaptés aux séries temporelles

- Détection de motifs locaux indépendamment de leur position (ex. descente, creux, remontée d'une goutte).
- Partage des poids → apprentissage efficace même avec peu de données.
- Possibilité de contrôler le champ réceptif en empilant plusieurs couches ou en utilisant des convolutions dilatées.

B.4.1 Limites des CNN 1D

- Champ réceptif linéaire avec la profondeur : pour capturer des dépendances à très long terme, il faut soit beaucoup de couches, soit des dilatations, soit une architecture différente (RNN, voir Annexe C).
- Sensibilité aux hyperparamètres : taille de noyau, stride, padding mal choisis peuvent perdre l'information.
- Moins adaptés aux séquences où l'ordre global est crucial (ex. prédiction pas à pas), car ils ne possèdent pas de mémoire d'état contrairement aux RNN.

B.5 Apprentissage des filtres

Contrairement à un filtre classique (Sobel, Gabor) fixé manuellement, les filtres d'un CNN sont appris par rétropropagation. Initialisés aléatoirement, ils se spécialisent progressivement :

- premières couches → détecteurs de bords, de pentes,
- couches profondes → motifs plus abstraits (formes, combinaisons de motifs).

Cela permet au réseau d'adapter ses détecteurs à la tâche spécifique (ici, les vallées de précipitations).

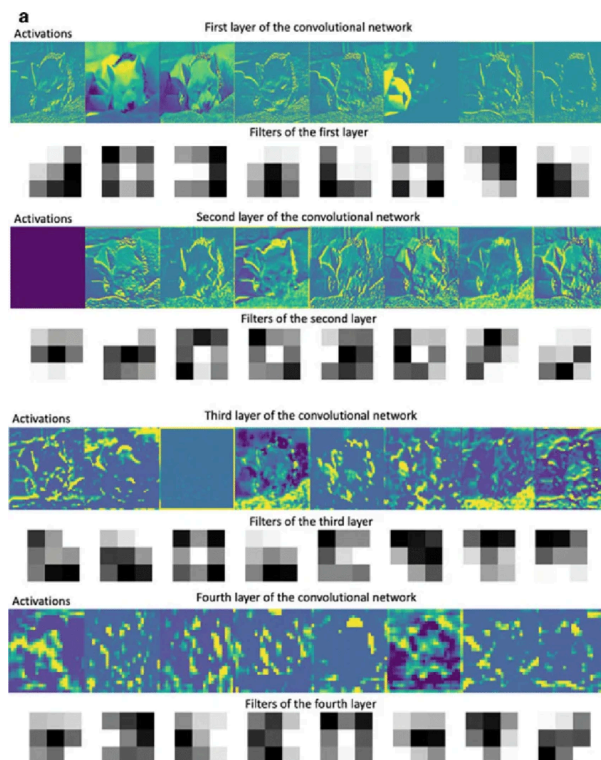


Figure B.5 – Visualisation des filtres appris dans les premières couches d'un CNN entraîné sur ImageNet (bords, couleurs, orientations).

B.6 Exemple d'architecture CNN 1D simple

```
nn.Conv1d(in_channels=1, out_channels=16, kernel_size=4, stride=2, padding=1)
nn.BatchNorm1d(16)
nn.ReLU()

nn.MaxPool1d(kernel_size=2, stride=2)
```

Cette séquence transforme un signal entrée (1 canal) en 16 cartes de caractéristiques, avec réduction de la longueur d'un facteur 4 (stride=2 puis pool=2).

Annexe C : Réseaux de neurones récurrents (RNN) pour la dynamique temporelle

C.1 Limites des CNN pour les séquences longues

Les réseaux convolutifs 1D (CNN 1D) utilisés dans l'auto-encodeur (chapitre 3) sont bien adaptés à l'extraction de motifs locaux (descente, creux, remontée). Cependant, ils présentent deux inconvénients pour modéliser l'évolution temporelle à long terme :

- Champ réceptif limité : la zone de l'entrée influençant une sortie croît linéairement avec le nombre de couches et la taille des filtres. Capturer des dépendances sur plusieurs secondes nécessiterait des réseaux très profonds ou des convolutions dilatées.
- Absence de mémoire d'état : chaque fenêtre est traitée indépendamment des précédentes. Le CNN ne possède pas de variable interne qui maintient une information d'un pas de temps à l'autre.

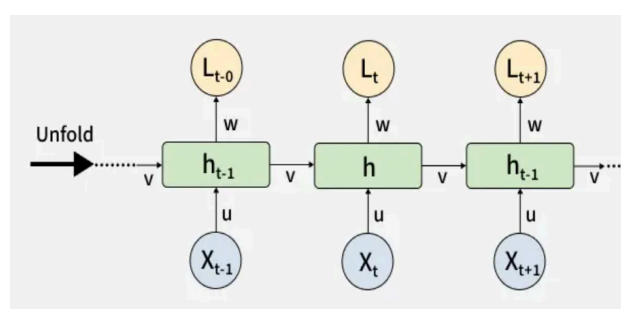
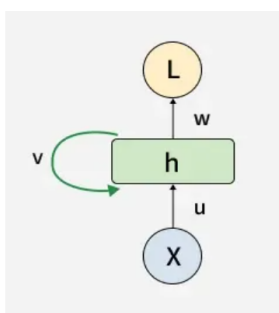
Pour remédier à ces limites, une approche complémentaire consiste à utiliser un réseau de neurones récurrent (RNN).

C.2 Principe du RNN et mémoire temporelle

Un RNN traite une séquence d'entrées (par exemple, les fenêtres temporelles successives) en maintenant un état caché noté h_t qui se propage d'un pas de temps au suivant. À chaque instant t , l'état caché est mis à jour à partir de l'entrée courante x_t et de l'état précédent h_{t-1} selon la formule :

$$h_t = \text{activation}(W_{xh} * x_t + W_{hh} * h_{t-1} + b_h)$$

où W_{xh} et W_{hh} sont des matrices de poids, b_h un biais, et l'activation est généralement tanh ou ReLU. Cette récurrence confère au RNN une mémoire théoriquement illimitée : l'information peut être propagée sur de nombreux pas de temps.



La Figure C.1 schématise le déroulement temporel d'un RNN.

C.3 Problèmes de gradient (vanishing / exploding)

L'entraînement des RNN utilise la rétropropagation à travers le temps (BPTT). Les gradients sont propagés sur toute la longueur de la séquence. Si les valeurs propres de la matrice de poids W_{hh} sont inférieures à 1, le gradient tend vers zéro (vanishing gradient) ; si elles sont supérieures à 1, il explose. Le vanishing gradient empêche l'apprentissage de dépendances à long terme (au-delà d'une dizaine de pas). C'est pourquoi des architectures plus complexes (LSTM, GRU) ont été développées.

C.4 Architectures avancées : LSTM et GRU

C.4.1 LSTM (Long Short-Term Memory)

La cellule LSTM introduit un état cellulaire c_t (mémoire à long terme) et trois portes qui contrôlent le flux d'information :

- Porte d'oubli (f_t) : décide quelles informations de l'état précédent doivent être oubliées.
- Porte d'entrée (i_t) : sélectionne les nouvelles informations à stocker.
- Porte de sortie (o_t) : filtre l'état cellulaire pour produire l'état caché h_t .

Ces mécanismes permettent au LSTM de conserver des informations sur des centaines de pas de temps (Figure C.2).

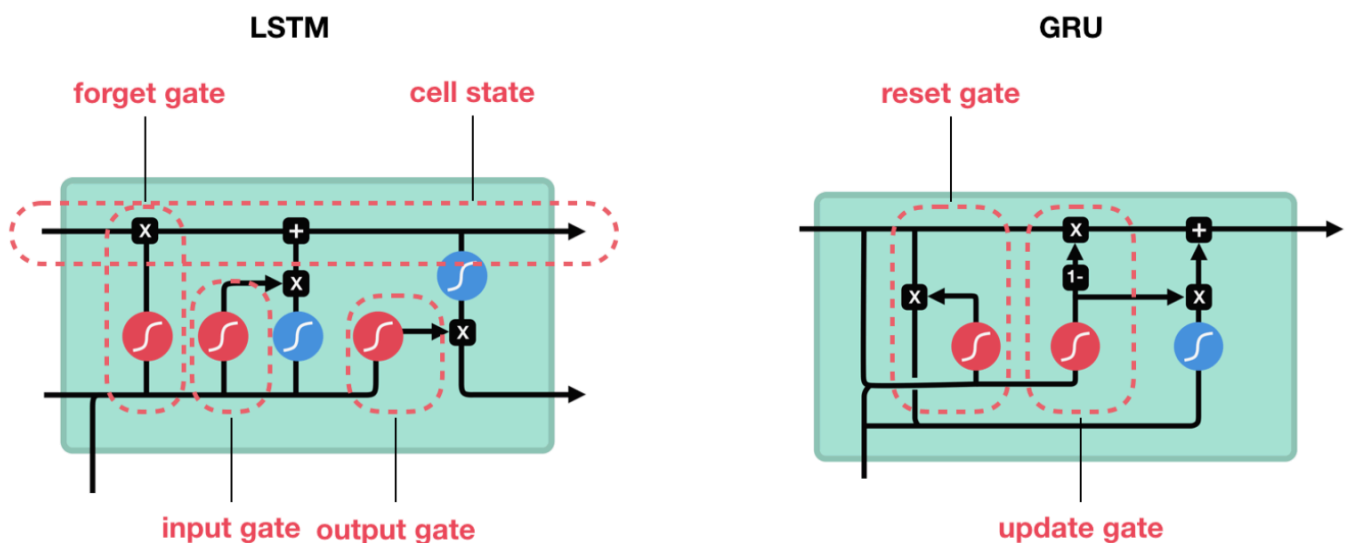


Figure C.2 – Architecture interne d'une cellule LSTM (portes d'oubli, entrée, sortie).

C.4.2 GRU (Gated Recurrent Unit)

Le GRU simplifie le LSTM en fusionnant la porte d'oubli et la porte d'entrée en une seule porte de mise à jour. Il n'a pas d'état cellulaire séparé ; l'état caché sert à la fois de mémoire et de sortie. Le GRU comporte moins de paramètres que le LSTM, ce qui le rend plus rapide, avec des performances souvent équivalentes.

C.5 Application envisagée dans le projet

L'objectif de l'étape 2 est d'extraire, pour chaque fenêtre temporelle, les caractéristiques physiques des gouttes : nombre de gouttes N , instants de passage t_i , diamètres D_i et vitesses V_i . Aucune vérité terrain n'est disponible (apprentissage non supervisé).

L'approche suggérée par l'encadrant consiste à utiliser un RNN (LSTM ou GRU) sur la séquence des vecteurs latents produits par l'auto-encodeur. Après l'entraînement du CAE (chapitre 3), chaque fenêtre de 512 points par voie est transformée en :

- un vecteur latent de dimension 64 (pour deux voies) ;
- la moyenne et l'écart-type de la fenêtre (2 valeurs par voie, soit 4 valeurs au total).

Au total, une fenêtre est représentée par 68 valeurs ($64 + 4$), au lieu de 1024 points bruts. Les moyennes et écarts-types sont sauvegardés car ils permettent de reconstruire le signal original (amplitudes absolues) et contiennent une information cruciale sur le diamètre des gouttes.

Le RNN prend en entrée la séquence des vecteurs latents $z_1, z_2, \dots, Z_t$ correspondant à des fenêtres successives (avec un pas de glissement de 256 points). Deux modes d'apprentissage sont envisagés :

- Auto-supervisé : le RNN prédit le vecteur latent suivant Z_{t+1} à partir des précédents. Il apprend ainsi la dynamique temporelle des précipitations.
- Extraction directe : à partir du signal reconstruit par l'AE (en utilisant les moyennes et écarts-types sauvegardés), on détecte les vallées, on calcule leur profondeur (liée au diamètre) et leur largeur (liée à la durée), et on estime le délai entre les deux voies pour obtenir la vitesse.

l'utilisation d'une ligne de référence (le niveau de repos du signal, ie: la moyenne de la fenêtre). La distance entre le signal et cette ligne de référence est directement liée à la présence et au diamètre d'une goutte. Un RNN pourrait être entraîné à minimiser l'erreur par rapport à cette ligne.

C.6 Contraintes et perspectives

- Absence de labels : l'évaluation des performances devra se faire sans vérité terrain, par des métriques non supervisées (erreur de prédiction du pas suivant, cohérence temporelle, ou validation visuelle).
- Délai variable entre voieBasse et voieHaute : le RNN devra être capable de capturer ce délai sans hypothèse a priori.
- Normalisation par fenêtre déjà appliquée dans l'AE – les moyennes et écarts-types sont conservés pour chaque fenêtre.
- Plusieurs gouttes simultanées : le modèle devra gérer des chevauchements dans une même fenêtre.

Annexe D : Codes sources

<https://github.com/otmaniamine/rnn-cnn-time-series.git>